

Backend Interview Q&A Guide

PatientCareApi Backend Interview Questions And Definitions

1. How would you describe this project?

PatientCareApi is an ASP.NET Core Web API for managing patients, treatments, and appointments. It uses clean architecture, EF Core with SQLite, DTOs, dependency injection, repository interfaces, Unit of Work, Swagger, and global error handling middleware.

2. What is a REST API?

A REST API is an HTTP-based API that exposes resources through URLs and uses HTTP methods to perform actions.

Common HTTP methods:

- * GET reads data.
- * POST creates data.
- * PUT updates an entire resource.
- * PATCH partially updates a resource or changes one state.
- * DELETE removes data.

In this project:

- * GET /api/patients gets patients.
- * POST /api/patients creates a patient.
- * PATCH /api/appointments/{id}/complete changes appointment state.

3. What is ASP.NET Core Web API?

ASP.NET Core Web API is a Microsoft framework for building HTTP APIs using C# and .NET.

It gives us:

- * Controllers
- * Routing
- * Middleware
- * Dependency injection
- * Configuration
- * Logging
- * Swagger/OpenAPI support

4. What is clean architecture?

Clean architecture separates code into layers with clear responsibilities.

The inner layer contains business rules.

The outer layers contain frameworks, databases, and API details.

In this project:

- * Domain contains business entities.
- * Application contains use cases.
- * Infrastructure contains EF Core and SQLite.
- * Api contains controllers and HTTP setup.

5. Why should Domain not depend on other projects?

Domain contains the core business rules.

It should not depend on frameworks like EF Core or ASP.NET Core because those are technical details.

This makes the domain easier to test, reuse, and change.

6. What is a DTO?

DTO means Data Transfer Object.

DTOs are used to move data between the client and the API.

Examples:

- * CreatePatientRequest
- * PatientResponse

Why use DTOs:

- * Avoid exposing domain entities directly.
- * Control API input and output.
- * Add validation attributes.
- * Keep API contracts stable.

7. Why not expose entities directly?

Exposing entities directly can cause problems:

- * Clients may see internal fields.
- * Database structure becomes tied to API response shape.
- * It is harder to change the domain model later.
- * Navigation properties can create large or circular responses.

DTOs protect the API boundary.

8. What is a controller?

A controller handles HTTP requests.

It should:

- * Receive request data.
- * Call a service.
- * Return HTTP response.

It should not:

- * Query DbContext directly.
- * Contain business rules.
- * Contain repeated try/catch error handling.

9. Why keep controllers thin?

Thin controllers are easier to read and test.

They keep HTTP logic separate from business logic.

In this project, controllers call service interfaces.

10. What is a service layer?

The service layer contains application workflows.

Example:

PatientService.CreateAsync validates the request, creates a Patient entity, adds it

through the repository, saves through Unit of Work, and returns a response DTO.

11. What is a repository?

A repository hides data access details behind an interface.

Application depends on:

IPatientRepository

Infrastructure implements:

PatientRepository

Why this matters:

The Application layer does not need to know EF Core details.

12. What is Unit of Work?

Unit of Work represents one commit/save operation.

Repositories stage changes.

UnitOfWork saves changes.

Example:

```
await _patientRepository.AddAsync(patient);  
await _unitOfWork.SaveChangesAsync();
```

Why it is useful:

If one service uses multiple repositories, it can save once after the full workflow succeeds.

13. What is EF Core?

Entity Framework Core is an Object Relational Mapper.

It lets C# code work with database records through C# classes.

Instead of writing SQL manually for every operation, EF Core tracks entities and translates LINQ queries into SQL.

14. What is DbContext?

DbContext represents a session with the database.

It tracks entities and saves changes.

In this project:

PatientCareDbContext has DbSet properties for Patients, Treatments, and Appointments.

15. What is DbSet?

DbSet represents a database table.

Example:

DbSet<Patient> Patients

This maps to the Patients table.

16. What is AsNoTracking?

AsNoTracking tells EF Core not to track returned entities.

Use it for read-only queries.

Benefits:

- * Faster reads.
- * Less memory usage.

Do not use it when you need to update the entity and save changes.

17. What is a tracked entity?

A tracked entity is monitored by EF Core.

If you change its properties and call `SaveChangesAsync`, EF Core knows what changed and updates the database.

In this project, `AppointmentRepository.GetByIdAsync` returns a tracked appointment so the service can call `Complete` or `Cancel`.

18. What is dependency injection?

Dependency injection means classes receive dependencies from the framework instead of creating them directly.

Example:

`PatientsController` receives `IPatientService`.

ASP.NET Core creates `PatientService` and injects it automatically.

Benefits:

- * Loose coupling.
- * Easier testing.
- * Easier replacement of implementations.

19. What is Program.cs?

`Program.cs` is the startup file.

It registers services, builds the app, configures middleware, maps controllers, and runs the API.

It is also the composition root because it connects interfaces to implementations.

20. What is middleware?

Middleware is code that runs in the HTTP request pipeline.

Examples:

- * HTTPS redirection
- * Authorization
- * Swagger
- * Error handling

Middleware can inspect, modify, stop, or pass along requests and responses.

21. What is global error handling?

Global error handling catches exceptions in one place and converts them into consistent HTTP responses.

In this project:

- * `NotFoundException` becomes 404.

- * ValidationException becomes 400.
- * ArgumentException becomes 400.
- * Unexpected exceptions become 500.

22. Why use custom NotFoundException?

It makes missing data explicit.

Instead of returning null everywhere, the service can say:

Patient with this id was not found.

The API middleware converts that exception into a 404 response.

23. What is validation?

Validation checks whether incoming data is acceptable.

Examples:

- * First name is required.
- * Date of birth must be in the past.
- * Email must be valid.
- * Treatment name is required.
- * Appointment reason is required.

Validation protects the system from bad input.

24. What is Swagger?

Swagger is an interactive API documentation tool.

It lets developers:

- * View endpoints.
- * Try requests.
- * See DTO schemas.
- * Test API behavior.

25. What is appsettings.json?

appsettings.json stores application configuration.

Examples:

- * Logging settings
- * Connection strings

The database path should be configured there instead of hardcoded inside DbContext.

26. What is a connection string?

A connection string tells the application how to connect to a database.

For SQLite:

Data Source=patientcare.db

This means the SQLite database file is patientcare.db.

27. What are migrations?

EF Core migrations are versioned database schema changes.

They let you create and update the database structure from your C# model.

Common commands:

dotnet ef migrations add InitialCreate

dotnet ef database update

28. What is authentication?

Authentication answers:

Who are you?

It verifies the identity of a user or system.

Examples:

- * Username and password
- * JWT token
- * Cookie login
- * OAuth login
- * API key

In a healthcare API, authentication is important because patient data is sensitive.

29. What is authorization?

Authorization answers:

What are you allowed to do?

It happens after authentication.

Example:

A doctor can view patient details.

A receptionist can create appointments.

A normal user cannot delete patient records.

30. Authentication vs Authorization

Authentication:

- * Confirms identity.
- * Example: User logs in and receives a token.

Authorization:

- * Checks permissions.
- * Example: User has Admin role and can delete records.

Simple explanation:

Authentication is who you are. Authorization is what you can access.

31. What is JWT?

JWT means JSON Web Token.

A JWT is a signed token that contains claims about a user.

Common claims:

- * User id
- * Email
- * Role
- * Expiration time

The client sends the token in the Authorization header:

Authorization: Bearer <token>

32. Why use JWT in APIs?

JWT is common for APIs because:

- * It works well with stateless APIs.
- * The server does not need to store session state.
- * It can include user claims.
- * It works with mobile apps, SPAs, and backend clients.

33. How would authentication be added to this project?

Possible steps:

1. Add an AuthController.
2. Add User entity or connect to Identity.
3. Validate username and password.
4. Generate JWT token.
5. Configure JWT Bearer authentication in Program.cs.
6. Add [Authorize] to protected controllers or actions.

34. How would authorization be added?

Possible approaches:

- * Role-based authorization
- * Policy-based authorization
- * Claim-based authorization

Example:

```
[Authorize(Roles = "Admin")]
```

This allows only users with the Admin role.

35. What is role-based authorization?

Role-based authorization checks the user's role.

Examples:

- * Admin
- * Doctor
- * Nurse
- * Receptionist

Example:

Only Admin can delete patients.

36. What is policy-based authorization?

Policy-based authorization uses named rules.

Example:

A policy named CanManageAppointments may require the user to be Doctor or Receptionist.

Policies are more flexible than simple roles.

37. Why is security important for PatientCareApi?

Patient data is sensitive.

A real healthcare API should protect:

- * Personal information
- * Medical treatments
- * Appointment history
- * Contact details

Important security ideas:

- * Use HTTPS.
- * Authenticate users.
- * Authorize actions.
- * Validate input.
- * Avoid leaking internal errors.
- * Log carefully.
- * Do not expose secrets in code.

38. What is HTTPS?

HTTPS encrypts communication between client and server.

It protects data in transit.

ASP.NET Core can redirect HTTP to HTTPS using `UseHttpsRedirection`.

39. What are HTTP status codes?

Status codes tell the client what happened.

Common codes:

- * 200 OK
- * 201 Created
- * 204 No Content
- * 400 Bad Request
- * 401 Unauthorized
- * 403 Forbidden
- * 404 Not Found
- * 500 Internal Server Error

40. Difference between 401 and 403

401 Unauthorized:

The user is not authenticated.

403 Forbidden:

The user is authenticated but not allowed to access the resource.

41. Why use async and await?

Database and network operations can take time.

Async/await allows the server to handle other requests while waiting.

This improves scalability.

42. What is CancellationToken?

`CancellationToken` lets a request be cancelled.

Example:

If the client disconnects, the API can stop the database operation.

This saves resources.

43. What is LINQ?

LINQ is Language Integrated Query.

It lets C# query collections and databases.

Example:

Where, OrderBy, Select, ToListAsync.

EF Core translates many LINQ queries into SQL.

44. What is cascade delete?

Cascade delete means related child records are deleted automatically when the parent is deleted.

In this project, deleting a Patient can delete related Treatments and Appointments.

This should be used carefully in real healthcare systems.

45. What is model binding?

Model binding is how ASP.NET Core converts HTTP request data into C# objects.

Example:

JSON body becomes CreatePatientRequest.

Route value {id} becomes int id.

46. What is [ApiController]?

[ApiController] enables helpful API behavior:

- * Automatic model validation response for invalid model state.
- * Better binding behavior.
- * Clearer API conventions.

47. What is CreatedAtAction?

CreatedAtAction returns 201 Created and includes information about where the new resource can be retrieved.

It is useful after POST create operations.

48. What is NoContent?

NoContent returns HTTP 204.

It means the operation succeeded but there is no response body.

It is common for DELETE operations.

49. What is a pull request?

A pull request is a request to merge changes from one branch into another.

It allows review, discussion, CI checks, and controlled merging.

50. What is CI/CD?

CI means Continuous Integration.

It automatically builds and tests code when changes are pushed.

CD means Continuous Delivery or Continuous Deployment.

It prepares or deploys the application after successful checks.

51. What is GitHub Actions?

GitHub Actions is GitHub's automation platform.

It can run:

- * dotnet restore
- * dotnet build
- * dotnet test
- * Docker build
- * Deployment steps

52. What is Docker?

Docker packages an application and its runtime dependencies into an image.

This makes the application run consistently across machines.

For this API, a Dockerfile can build and run the ASP.NET Core app in a container.

53. What is a Docker image?

A Docker image is a packaged version of the application.

It includes the app files and runtime environment.

Images can be pushed to registries like GitHub Container Registry.

54. What is a Docker container?

A container is a running instance of an image.

Image is like a blueprint.

Container is the running process.

55. What questions might a manager ask?

Question:

Why did you use clean architecture?

Answer:

To separate business logic from database and API details. It makes the project easier to test, maintain, and explain.

Question:

Why did you use DTOs?

Answer:

To control API input and output and avoid exposing internal domain or database models directly.

Question:

Why did you use services?

Answer:

Services hold application workflows so controllers stay thin.

Question:

Why did you use repositories?

Answer:

Repositories hide EF Core details from the Application layer.

Question:

Why did you use Unit of Work?

Answer:

To save all pending changes once after a workflow completes.

Question:

How do you handle errors?

Answer:

Services throw meaningful exceptions, and global middleware converts them to consistent HTTP responses.

Question:

How would you secure this API?

Answer:

I would add authentication using JWT or ASP.NET Core Identity, then use authorization roles or policies to restrict endpoints.

56. Strong Final Interview Summary

I built PatientCareApi as a layered ASP.NET Core Web API. I used Domain entities for business rules, Application services for workflows, repository interfaces for persistence abstractions, Infrastructure with EF Core and SQLite for database access, and thin API controllers for HTTP endpoints. I also added Swagger, global error handling, and a GitHub feature-branch workflow. The next steps are migrations, tests, CI, and Docker.